Guía de despliegue en Hostinger — Tienda TodoAki

Cómo pasar la tienda de tu computador a internet, con un hosting y un dominio de Hostinger.

> **Lee primero esto.** El proyecto son **dos aplicaciones**: - **WordPress + WooCommerce** (PHP + MySQL) -> el panel de administración y la API. Hostinger lo corre nativamente. - **Next.js** (Node.js) -> la tienda que ve el cliente. Node **no** corre en el hosting compartido normal de Hostinger; necesita un **VPS** o ponerlo en **Vercel**.
>
> Por eso hay que decidir **dónde va el frontend** antes de empezar.

---

1. Decidir el plan (la decisión clave)

Opción	WordPress	Next.js (front)	Dificultad	Recomendado para
A — Híbrida (recomendada)	Hostinger (plan WordPress/Premium/Business)	**Vercel** (gratis o ~US$20/mes)	Baja	Empezar rápido y sin dolores con Node
B — Todo en Hostinger	Hostinger **VPS**	Hostinger **VPS** (Node + PM2 + Nginx)	Media/Alta	Quien quiere todo en un solo proveedor

Recomendación: empieza con la **Opción A**. Vercel está hecho por el equipo de Next.js: despliega el front conectando tu repo de GitHub, sin configurar servidores. Dejas WordPress en el hosting de Hostinger (que es justo para lo que sirve) y el front en Vercel. Más adelante, si quieres todo junto, migras a la Opción B.

Ambas opciones comparten **toda la Parte 2 (WordPress)**. Cambian solo en la **Parte 3 (frontend)**.

---

2. Qué comprar

- **Dominio:** por ejemplo `tutienda.com` (en Hostinger o donde prefieras).
- **Hosting:**
- **Opción A:** un plan **Premium / Business de WordPress** de Hostinger (trae WordPress en 1 clic, MySQL y SSL gratis). El front irá en Vercel (cuenta aparte, gratis para empezar).
- **Opción B:** un **VPS** de Hostinger (KVM 1 o superior, con Ubuntu). Ahí correrán WordPress **y** Next.js.

Cómo repartiremos el dominio

- `tutienda.com` -> **la tienda** (Next.js). Es la que comparte el cliente.
- `admin.tutienda.com` -> **WordPress** (subdominio). Es tu panel y la API.

Esto se configura con registros DNS en hPanel (Parte 4).

---

3. Parte 2 — WordPress + WooCommerce (igual en A y B)

3.1. Instalar WordPress

- **Opción A:** en hPanel -> *Sitios web* -> *Añadir sitio* -> **WordPress**, y asócialo al subdominio `admin.tutienda.com`.
- **Opción B (VPS):** instala la plantilla de WordPress del VPS, o un stack LAMP + WordPress manual.

Entra a `https://admin.tutienda.com/wp-admin` con el usuario admin que creaste.

3.2. Instalar WooCommerce

En wp-admin -> *Plugins* -> *Añadir nuevo* -> busca **WooCommerce** -> Instalar y Activar. En el asistente: país **Colombia**, moneda **peso colombiano (COP)**.

3.3. Subir el plugin propio `tienda-headless`

1. En tu computador, comprime la carpeta `wordpress-plugin/tienda-headless` en un `.zip` (que el zip contenga la carpeta `tienda-headless`, no su interior suelto).
2. wp-admin -> *Plugins* -> *Añadir nuevo* -> *Subir plugin* -> elige el `.zip` -> Instalar -> **Activar**.

3.4. Ajustes de WooCommerce

- *WooCommerce -> Ajustes -> General:* moneda **COP**, país de venta **Colombia**.
- *WooCommerce -> Ajustes -> Envío:* crea una zona de envío para Colombia con tus tarifas.
- *Ajustes -> Enlaces permanentes:* elige **Nombre de la entrada** y guarda (necesario para que la API REST responda bien).

3.5. Configurar la pasarela Wompi (headless)

WooCommerce -> Ajustes -> Pagos -> Wompi (headless):

Campo	Valor
Llave pública	`pub_test_…` (sandbox) o `pub_prod_…` (producción)
Secreto de integridad	el de Wompi -> Desarrolladores
Secreto de eventos	el de Wompi -> Desarrolladores
URL de la tienda (Next.js)	`https://tutienda.com` (¡la del front, no la de WordPress!)
Secreto de revalidación	un texto largo y aleatorio — **guárdalo**, va igual en el front
Activar	

> El "Secreto de revalidación" debe ser **idéntico** al `REVALIDATE_SECRET` del front (Parte 3).

3.6. Subir el catálogo

Ya tienes los productos importados en tu WordPress local; la forma más fácil de llevarlos es **migrar la base de datos**:

- Instala el plugin **All-in-One WP Migration** en el WordPress **local** (el de Docker) y en el de Hostinger.
- En el local: *Exportar -> a archivo*. En Hostinger: *Importar* ese archivo.
- Esto lleva productos, categorías, imágenes y ajustes de una vez.

Alternativa (instalación limpia): volver a correr los scripts de importación desde Shopify en el servidor, con WP-CLI (`wp eval-file`), poniendo `SHOPIFY_STORE` y `SHOPIFY_TOKEN` temporalmente. Solo si tienes acceso SSH/WP-CLI.

3.7. SSL

En hPanel activa el **SSL gratis** para `admin.tutienda.com` (suele ser automático). Todo debe quedar en `https://`.

---

4. Parte 3 — El frontend (Next.js)

Variables de entorno del front (las dos opciones)

Variable	Valor en producción
`WC_STORE_URL`	`https://admin.tutienda.com` (sin `/` al final)
`WOMPI_ENV`	`production` (o `sandbox` mientras pruebas)
`REVALIDATE_SECRET`	el **mismo** texto que pusiste en la pasarela (3.5)

> El **token de Shopify NO va aquí** — solo se usa para importar el catálogo (Parte 3.6), nunca en la tienda en vivo.
>
> **Orden importante:** despliega y configura **WordPress primero**, con productos cargados. El `build` del front pre-genera el catálogo y necesita que WordPress ya responda.

Opción A — Vercel (recomendada)

1. Crea una cuenta en vercel.com y conéctala con tu GitHub.
2. *Add New -> Project* -> importa el repo **`Slaternight/TodoAki`**.
3. En la configuración del proyecto: - **Root Directory:** `frontend` - Framework: Next.js (se detecta solo).
4. En *Settings -> Environment Variables* agrega las tres variables de arriba.
5. **Deploy.** Vercel te da una URL tipo `todoaki.vercel.app`.
6. En *Settings -> Domains* añade `tutienda.com` (Vercel te dirá qué registro DNS crear en Hostinger — ver Parte 5).

Cada vez que hagas `git push`, Vercel reconstruye y publica solo.

Opción B — VPS de Hostinger (Node + PM2 + Nginx)

En el VPS (Ubuntu), por SSH:

```
# 1. Node 20 (si no está)
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# 2. Traer el código
git clone https://github.com/Slaternight/TodoAki.git
cd TodoAki/frontend

# 3. Variables de entorno
cp .env.example .env.local
nano .env.local        # pon WC_STORE_URL, WOMPI_ENV, REVALIDATE_SECRET

# 4. Instalar y construir
npm ci
npm run build

# 5. Mantenerlo corriendo con PM2
sudo npm install -g pm2
pm2 start "npm run start" --name tienda      # Next escucha en el puerto 3000
pm2 save && pm2 startup                       # para que reviva al reiniciar el VPS
```

Luego **Nginx** como proxy inverso de `tutienda.com` al puerto 3000:

```
server {
  server_name tutienda.com www.tutienda.com;
  location / {
    proxy_pass http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
  }
}
```

Y el **SSL** con Let's Encrypt:

```
sudo apt-get install -y certbot python3-certbot-nginx
sudo certbot --nginx -d tutienda.com -d www.tutienda.com
```

Para actualizar en el futuro: `git pull && npm ci && npm run build && pm2 restart tienda`.

---

5. Parte 4 — Conectar el dominio (DNS en hPanel)

En hPanel -> *Dominios* -> *Zona DNS* de `tutienda.com`:

Para	Registro	Nombre	Apunta a
WordPress (admin)	A	`admin`	la **IP del hosting/VPS** de WordPress
Tienda (Opción A, Vercel)	según indique Vercel	`@` y `www`	el destino que te dé Vercel (CNAME/A)
Tienda (Opción B, VPS)	A	`@` y `www`	la **IP del VPS**

Los cambios de DNS pueden tardar desde minutos hasta unas horas en propagarse.

---

6. Parte 5 — Pasar Wompi a producción

Cuando ya pruebes con sandbox y quieras cobrar de verdad:

1. En la pasarela (3.5) cambia las llaves a las de **producción** (`pub_prod_…`) y sus secretos.
2. En el front, pon `WOMPI_ENV=production` (en Vercel: Environment Variables -> redeploy; en VPS: edita `.env.local` y `pm2 restart tienda`).
3. En el panel de **Wompi -> Desarrolladores**, configura la **URL de eventos** (webhook): `https://admin.tutienda.com/wp-json/tienda-headless/v1/wompi/events` Esa URL debe ser pública y con HTTPS (en producción ya lo es; el truco del túnel `cloudflared` era solo para local).

---

7. Checklist final

- [ ] `https://admin.tutienda.com/wp-admin` abre y WooCommerce está activo.
- [ ] El plugin **tienda-headless** está activo y configurado (llaves, URL de la tienda, secreto de revalidación).
- [ ] Hay productos en el catálogo.
- [ ] `https://tutienda.com` muestra la tienda con los productos.

- [ ] `WC_STORE_URL`, `WOMPI_ENV` y `REVALIDATE_SECRET` están puestos en el front y el secreto coincide con el del plugin.
- [ ] Al editar un producto en wp-admin, el catálogo del front se actualiza (la revalidación llega).
- [ ] Una compra de prueba (sandbox) redirige a Wompi y vuelve a `/checkout/resultado`, y el pedido queda registrado (webhook).
- [ ] Registro, login, "mis pedidos" y recuperar contraseña funcionan (el correo de recuperación necesita que el envío de correos del hosting funcione; en Hostinger suele venir listo).
- [ ] Todo en `https://` (SSL en ambos).

---

8. Seguridad (no te saltes esto)

- **Rotar el token de Shopify** antes de usarlo en producción (estuvo expuesto durante el desarrollo). Ponlo solo donde corra la importación, nunca en el repo ni en el front.
- Los **secretos** viven solo del lado servidor: los de Wompi y el de revalidación en los ajustes de WordPress (en su base de datos) y en las variables del front; nunca en el código ni en el navegador.
- En producción **no** definas la constante `TH_REVALIDATE_URL` en `wp-config.php` (era solo para Docker con `host.docker.internal`): así el plugin usa automáticamente `URL de la tienda + /api/revalidate`.
- Mantén WordPress, WooCommerce y los plugins actualizados.

Arquitectura de referencia: `tutienda.com` (Next.js) <-> `admin.tutienda.com` (WordPress/WooCommerce + plugin Wompi) <-> Wompi (pagos). El navegador solo habla con la tienda; la tienda habla con WordPress por detrás.